1) Project Planning & Management

---

1. Project Proposal

Project Title

Skill Swap Platform with Points System, Video Call, and Chatbot Integration

---

Project Overview

The Skill Swap platform is a mobile application that enables users to exchange skills using a points-based system instead of money. Each user can teach or learn skills and set their own hourly rate in points. The system manages session requests, scheduling, secure point transactions, and communication between users through chat functionality.

Additionally, the system integrates video call functionality to allow users to conduct learning sessions remotely. A chatbot is also integrated using an API to assist users, answer questions, and provide guidance in the app, users can purchase points using a **simulated payment process** (for demonstration purposes)

The application aims to create a collaborative learning environment where users can share knowledge, gain new skills, and build a trusted community without financial barriers.

Project Objectives

The main objectives of this project are:

- Develop a mobile application for skill exchange
- Implement a dynamic points wallet system
- Allow users to set their own hourly price in points
- Enable users to request and manage learning sessions
- Provide real-time chat between users
- Integrate video call functionality for online sessions
- Integrate a chatbot using API to assist users
- Ensure secure and reliable data management
- Create a user-friendly and modern interface

Project Scope

Included in Scope

The system will include:

- User registration and login
- Authentication system
- User profile management
- Adding and managing skills
- Setting price per hour in points
- Home dashboard display
- Searching for skills
- Requesting sessions
- Accepting or rejecting session requests
- Points wallet management
- Real-time chat system
- Video call integration for conducting sessions
- Chatbot integration using API
- Rating and review system
- Notifications

Excluded from Scope

The system will NOT include:

- Real money payments
- External payment gateways
- Advanced AI recommendation engine
- Physical meeting management system

---

2) Project Plan

Project Timeline (10 Weeks Example)

Week	Phase	Deliverable
Week 1	Requirements Gathering	User stories and requirements

Week	Phase	Deliverable
Week 2	System Analysis	Use case diagram
Week 3	Database Design	ER diagram
Week 4	UI/UX Design	Wireframes and mockups
Week 5–7	Implementation	Core system development
Week 8	Integration	Feature integration
Week 9	Testing	System testing
Week 10	Final Delivery	Documentation and presentation

Milestones

- Requirements Completed
- System Design Completed
- Database Design Completed
- UI/UX Design Completed
- Core Features Implemented
- System Integration Completed
- Testing Completed
- Final Submission

Deliverables

The project will deliver:

- Mobile Application
- Firebase Database
- System Documentation
- UI/UX Design
- Test Results
- Final Presentation

Resource Allocation

Resource	Description
Team Members	4 Students
Development Tools	Flutter, Firebase
Hardware	Personal laptops

Resource	Description
Internet	Required for cloud services

---

3) Task Assignment & Roles

Team Structure

The project team consists of four members. Each member is responsible for developing a complete feature, including both frontend and backend components. This feature-based development approach ensures clear responsibility, faster development, and easier testing.

Member 1 — Authentication & Chat Feature Developer

Responsibilities:

- Implement user registration and login system
- Manage authentication using Firebase
- Handle password reset functionality
- Develop real-time chat system
- Implement message sending and receiving
- Store chat data in Firestore database
- Ensure secure user authentication

Assigned Feature:

Authentication & Chat System

Member 2 — Home Feature Developer

Responsibilities:

- Design and implement Home screen interface
- Display user wallet balance
- Show recommended skills
- Display upcoming sessions
- Handle navigation between screens
- Fetch data from database
- Ensure responsive UI

Assigned Feature:

Home / Dashboard System

Member 3 — Sessions Feature Developer

Responsibilities:

- Implement session request functionality
- Handle session scheduling
- Manage session acceptance and rejection
- Calculate session cost based on points
- Update session status
- Trigger points transfer after session completion
- Manage video call session start and end

Assigned Feature:

Sessions Management System

Member 4 — Profile Feature Developer

Responsibilities:

- Implement profile interface
- Manage profile editing
- Handle skills management (Add / Edit / Delete)
- Allow users to set price per hour in points
- Display user ratings and reviews
- Store and update profile data

Assigned Feature:

Profile Management System

---

4) Risk Assessment & Mitigation Plan

Risk	Description	Mitigation Solution
System crash	Application stops working	Implement error handling and regular testing
Data loss	User data may be lost	Enable Firebase automatic backup
Security breach	Unauthorized access	Use secure authentication and database rules
Internet failure	User loses connection	Implement retry mechanism
API failure	Chatbot or video call service not responding	Handle API errors and show fallback message
Delayed development	Tasks take longer than planned	Track weekly progress
Integration issues	Features do not work together	Perform integration testing

---

5) KPIs (Key Performance Indicators)

Performance Metrics

System Response Time

The system should respond within:

Less than **2 seconds**

System Uptime

The system should be available:

At least **99% uptime**

Session Success Rate

Percentage of sessions successfully completed:

Greater than **90%**

User Adoption Rate

Number of users who register and use the system during testing:

Target: **50 users**

Payment System Note:

The payment functionality in this system is implemented as a simulated payment process for demonstration purposes. No real financial transactions are processed. The system mimics payment confirmation and updates the user's points balance accordingly.

---

2) Literature Review

Feedback & Evaluation

The project was evaluated by the lecturer based on its concept, feasibility, and implementation approach. The overall feedback highlighted that the idea of a skill exchange platform using a points-based system is innovative and practical.

The integration of multiple features such as dynamic pricing, real-time chat, video call functionality, and chatbot support was positively received, as it demonstrates the ability to design a comprehensive and modern application.

The lecturer also emphasized the importance of maintaining system reliability, ensuring smooth user experience, and implementing secure data handling practices.

Overall, the evaluation indicated that the project has strong potential as a complete system, provided that all components are properly integrated and tested.

Suggested Improvements

Based on the lecturer's feedback, the following improvements were suggested:

- Enhance the user interface to ensure simplicity and ease of use
- Improve system performance to reduce response time
- Add better error handling for edge cases (e.g., failed sessions, connection issues)
- Ensure accurate calculation and handling of points transactions
- Improve the chatbot responses to provide more helpful and relevant assistance
- Enhance the recommendation system for skills (optional future improvement)
- Add more validation to prevent incorrect user inputs
- Improve testing coverage to ensure system stability

These improvements aim to enhance both the technical quality and the user experience of the system.

Final Grading Criteria

The project will be evaluated based on the following criteria:

1. Documentation (25%)

- Clarity and organization of the report
- Completeness of all required sections
- Proper explanation of system design and architecture

---

2. Implementation (35%)

- Functionality of core features
- Integration between frontend and backend
- Correct implementation of the points system
- Performance and responsiveness of the application

---

3. Testing & Validation (20%)

- Unit testing of individual components
- Integration testing between system modules

- User acceptance testing
- Handling of errors and edge cases

---

4. Presentation (20%)

- Clarity of explanation during presentation
- Demonstration of system features
- Ability to answer questions
- Overall professionalism

---

3) Requirements Gathering

Stakeholder Analysis

The system involves several stakeholders, each with specific needs and expectations:

1. Users (Learners & Tutors)

- Exchange skills with other users
- Set their own hourly rate in points
- Book and attend sessions
- Communicate through chat and video calls
- Track their points balance
- Purchase additional points when needed

---

2. System Administrator (Optional Role)

- Monitor system performance
- Manage user accounts if necessary
- Handle reports and issues
- Ensure data consistency and system reliability

---

3. Development Team

- Design and implement system features
- Ensure smooth integration between frontend and backend
- Maintain system performance and scalability
- Fix bugs and improve the system

---

4. Lecturer / Evaluator

- Evaluate the system based on functionality and documentation
- Ensure that the project meets academic requirements
- Assess system design, implementation, and presentation

User Stories & Use Cases

User Stories

- As a user, I want to create an account so that I can use the platform
- As a user, I want to log in securely to access my profile
- As a user, I want to add my skills and set my price in points
- As a user, I want to browse other users' skills
- As a user, I want to request a session with another user
- As a user, I want to accept or reject session requests
- As a user, I want to chat with other users in real-time
- As a user, I want to have a video call during sessions
- As a user, I want to earn and spend points
- As a user, I want to buy points using a simulated payment system
- As a user, I want to interact with a chatbot for assistance

---

Use Cases

1. User Registration & Login

- User enters required information
- System validates input
- Account is created or access is granted

---

2. Manage Profile

- User adds or edits skills
- User sets price per hour
- System updates profile data

---

3. Session Booking

- User selects a skill provider
- Sends session request
- Provider accepts or rejects
- Session is scheduled

---

4. Points Transaction

- Points are deducted from requester
- Points are added to provider after session completion

---

5. Buy Points (Simulated Payment)

- User selects points package
- Confirms payment
- System adds points to wallet

---

6. Chat & Video Call

- Users exchange messages in real-time
- Users start video call during session

---

7. Chatbot Interaction

- User sends a question
- Chatbot processes request via API
- Chatbot returns a response

Functional Requirements

The system shall:

- Allow users to register and log in securely
- Allow users to manage their profiles and skills
- Allow users to set their own hourly rate in points
- Allow users to browse available skills and providers
- Allow users to request, accept, and reject sessions
- Allow users to communicate via real-time chat
- Allow users to conduct video calls
- Allow the system to calculate and transfer points between users
- Allow users to view their wallet balance
- Allow users to purchase points using a simulated payment process
- Integrate a chatbot using an external API
- Store and retrieve data using a database system

Non-functional Requirements

1. Performance

- The system should respond to user actions within 2 seconds
- Real-time features (chat & video) should have minimal latency

---

2. Security

- User authentication must be secure
- User data must be protected
- Unauthorized access must be prevented

---

3. Usability

- The interface should be simple and intuitive
- Users should be able to navigate easily
- The system should be responsive on different devices

---

4. Reliability

- The system should be available معظم الوقت (high uptime)
- The system should handle errors without crashing
- Data should be stored consistently without loss

---

5. Scalability

- The system should handle increasing number of users
- The database should support growth in data حجم

---

4) System Analysis & Design

---

1. Problem Statement & Objectives

Problem Statement

Many individuals have valuable skills but lack an accessible platform to exchange knowledge without relying on money. Traditional learning platforms often require financial payment, which limits accessibility for users who want to learn but cannot afford paid courses.

Additionally, existing platforms rarely support mutual skill exchange where users can both teach and learn, while also managing value through a flexible system like points.

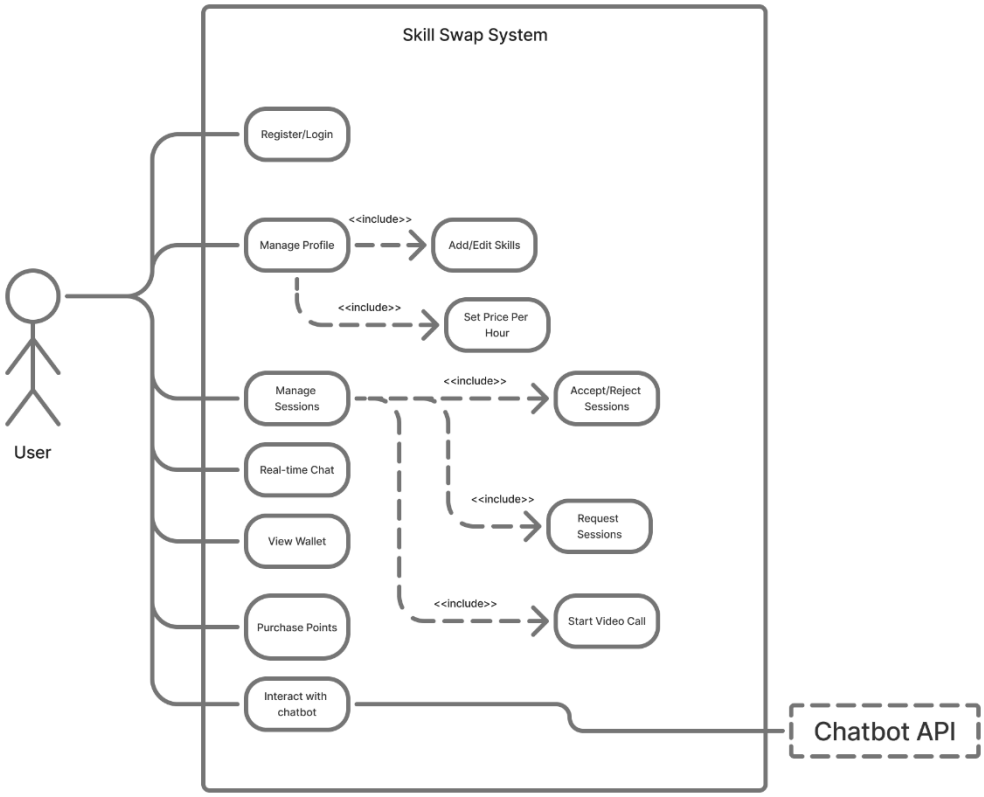
Objectives

The main objectives of this project are:

- To develop a platform that enables users to exchange skills using a points-based system
- To allow users to define the value of their time by setting their own hourly rate
- To provide real-time communication through chat and video calls
- To enable users to manage sessions بسهولة (booking, accepting, completing)
- To integrate a chatbot for user assistance and guidance
- To simulate a payment system for purchasing points (for demonstration purposes)
- To ensure a smooth, secure, and user-friendly experience

Use Case Diagram & Descriptions

Diagram



Actors

- **User** (Primary Actor)
- **Chatbot API** (External System)

Main Use Cases

- Register / Login
- Manage Profile
- Add Skills & Set Price
- Browse Skills
- Request Session
- Accept / Reject Session
- Chat with Users
- Start Video Call
- Manage Points Wallet
- Buy Points (Simulated Payment)
- Interact with Chatbot

Use Case Description 1 — Register / Login

Use Case Name: Register / Login

Actor: User

Description:
Allows users to create a new account or log in to an existing account.

Precondition:
User has access to the application.

Main Flow:

1. User opens the application
2. User enters email and password
3. System verifies credentials
4. User is logged into the system

Postcondition:
User successfully accesses the system.

Use Case Description 2 — Manage Profile

Use Case Name: Manage Profile

Actor: User

Description:
Allows users to create, update, and manage their personal profile information.

Precondition:
User is logged into the system.

Main Flow:

1. User opens profile page
2. User edits personal information
3. User saves changes
4. System updates profile data

Postcondition:
Profile information is updated successfully.

Use Case Description 3 — Add / Edit Skills

Use Case Name: Add/Edit Skills

Actor: User

Description:
Allows users to add new skills or modify existing skills.

Precondition:
User is logged in.

Main Flow:

1. User opens skills section
2. User adds or edits skill details
3. User saves changes
4. System stores skill data

Postcondition:
Skill information is updated.

Use Case Description 4 — Request Session

Use Case Name: Request Session

Actor: User

Description:
Allows users to request a learning session with another user.

Precondition:
User has enough points.

Main Flow:

1. User selects a skill
2. User chooses session time
3. User submits request
4. System sends request notification

Postcondition:
Session request is created.

Use Case Description 5 — Accept / Reject Session

Use Case Name: Accept/Reject Session

Actor: User

Description:
Allows users to respond to session requests.

Precondition:
A session request exists.

Main Flow:

1. User receives session request
2. User reviews request
3. User accepts or rejects request
4. System updates session status

Postcondition:
Session status is updated.

Use Case Description 6 — Purchase Points

Use Case Name: Purchase Points

Actor: User

Description:
Allows users to buy points to use for booking sessions.

Precondition:
User is logged into the system.

Main Flow:

1. User selects points package
2. User confirms payment
3. System processes transaction
4. System updates wallet balance

Postcondition:
Points are added to the user's wallet.

Use Case Description 7 — Interact with Chatbot

Use Case Name: Interact with Chatbot

Actor: User

External System: Chatbot API

Description:
Allows users to ask questions and receive automated assistance.

Precondition:
User is logged in.

Main Flow:

1. User opens chatbot
2. User sends message
3. Chatbot processes request
4. Chatbot sends response

Postcondition:
User receives assistance.

Functional & Non-Functional Requirements

Functional Requirements

The system shall:

- Allow user registration and authentication
- Allow users to manage profiles and skills
- Allow users to set price per hour in points
- Allow users to request and manage sessions
- Allow real-time chat between users
- Allow video call functionality during sessions
- Handle points transactions between users
- Allow users to purchase points via simulated payment
- Integrate chatbot functionality via API

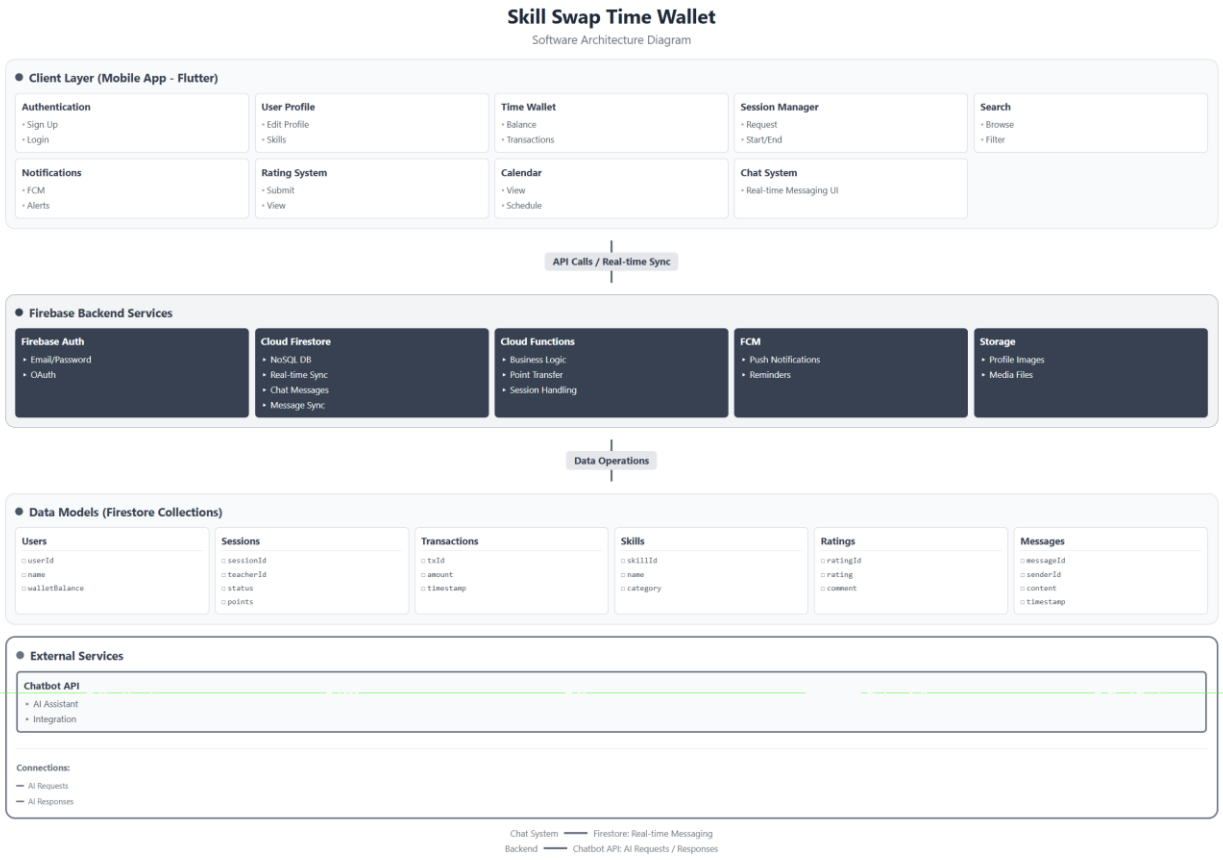
Non-Functional Requirements

- **Performance:** Fast response time and low latency for real-time features
- **Security:** Secure authentication and data protection
- **Usability:** Simple and intuitive user interface
- **Reliability:** High system availability and stability
- **Scalability:** Ability to handle increasing users and data

---

Software Architecture

Diagram



Architecture Style

The system follows a **Client–Server Architecture** using a **layered design approach**.
The mobile application acts as the client, while Firebase cloud services handle backend logic, data storage, and real-time operations.

The architecture is divided into four main layers:

- Client Layer
- Backend Services
- Data Layer
- External Services

This separation improves scalability, maintainability, and system performance.

High-Level System Components

1) Client Layer (Mobile Application – Flutter)

The client layer represents the user interface and handles all user interactions.

Its main responsibilities include:

- Displaying the user interface
- Handling user input
- Communicating with backend services
- Managing navigation between screens

Main modules shown in the diagram:

- Authentication
- User Profile
- Time Wallet
- Session Manager
- Search
- Notifications
- Rating System
- Calendar
- Chat System

---

2) Backend Services (Firebase)

The backend layer is responsible for processing application logic and managing real-time data.

It includes:

- **Firebase Authentication** → user identity management
- **Cloud Firestore** → real-time database and chat storage
- **Cloud Functions** → business logic (sessions, points transfer)
- **Firebase Cloud Messaging (FCM)** → notifications and reminders
- **Firebase Storage** → media files (e.g., profile images)

---

3) Data Layer (Firestore Collections)

The data layer stores application data in structured collections.

Main collections include:

- Users
- Sessions
- Transactions
- Skills
- Ratings
- Messages

This structure supports efficient data retrieval and real-time updates.

---

4) External Services

External systems extend the application functionality.

- **Chatbot API** → provides automated responses and user assistance

---

System Interaction Flow

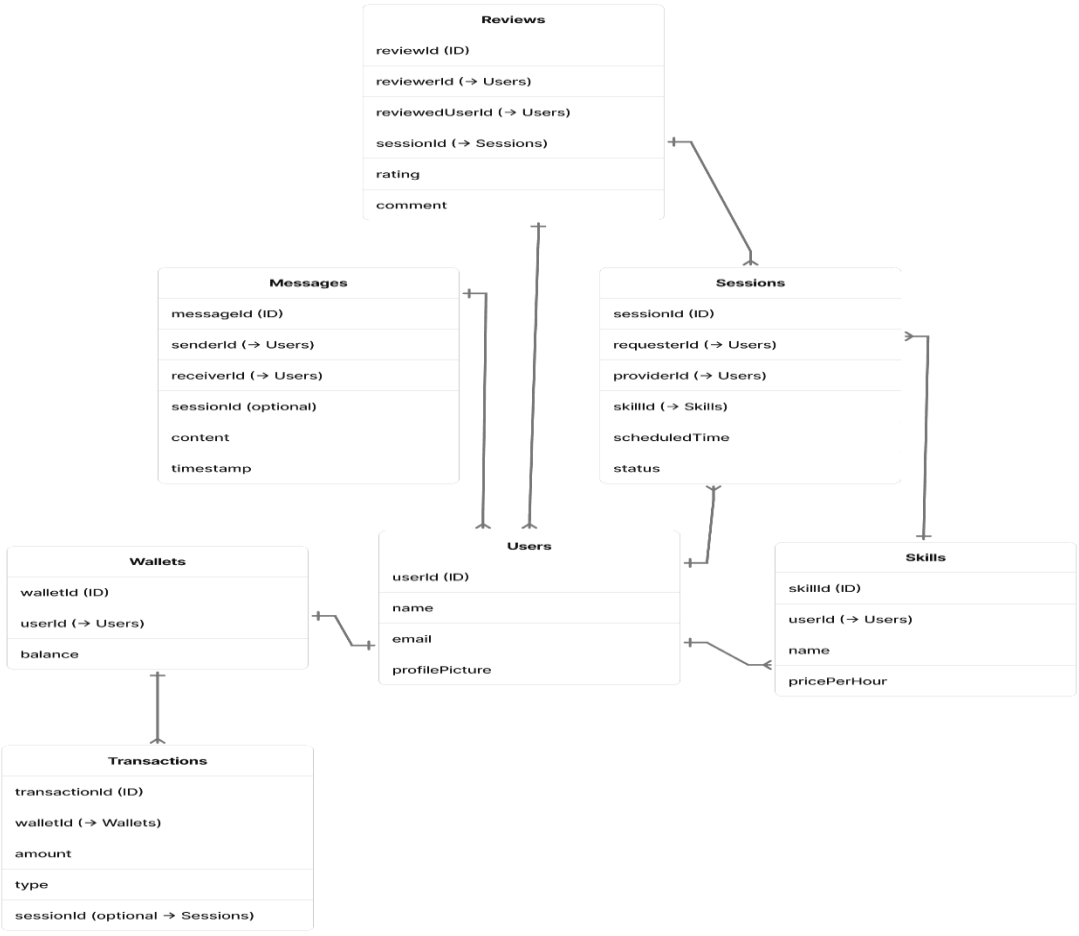
1. The user interacts with the mobile application
2. The client sends requests to backend services
3. The backend processes logic and interacts with the database
4. Data is stored or retrieved from Firestore
5. The backend returns the response to the client
6. External services (Chatbot API) are called when required

Key Notes

- Real-time features (chat and updates) are supported by Firestore
- Business logic is handled through Cloud Functions
- The system uses a modular layered structure for clarity and scalability

Entity Relationship Diagram Description

Diagram



The system uses a NoSQL database (Firebase Firestore), where data is organized into collections and documents. Relationships between entities are represented using document references rather than traditional foreign keys.

The main collections in the system include Users, Skills, Sessions, Wallets, Transactions, Messages, and Reviews.

The **Users** collection represents the core entity of the system and is connected to multiple components. Each user can create multiple skills, participate in multiple sessions as a requester or provider, send and receive messages, and write or receive reviews.

The **Skills** collection stores the skills offered by users. Each skill belongs to one user, while a user can have multiple skills, forming a one-to-many relationship.

The **Sessions** collection represents learning sessions between users. Each session is associated with one requester, one provider, and one skill. A user can participate in many sessions.

The **Wallets** collection manages user balances. Each user has exactly one wallet, forming a one-to-one relationship. The wallet is linked to the **Transactions** collection, where each wallet can have multiple transactions representing point transfers or purchases.

The **Messages** collection supports communication between users. Each message is linked to a sender and receiver and may optionally be associated with a session.

The **Reviews** collection stores feedback between users after sessions. Each review is linked to a reviewer, a reviewed user, and a session.

Overall, the data model is designed to support scalability and real-time updates, while maintaining clear relationships between system components using document references.

Logical Schema

The system uses a NoSQL data model implemented using Firebase Firestore. Data is organized into collections and documents, where each collection represents a main entity in the system.

The logical schema consists of the following collections:

Users

- userId (Primary Identifier)
- name
- email
- profilePicture
- walletBalance

Skills

- skillId (Primary Identifier)
- userId (Reference to Users)
- name
- pricePerHour

Sessions

- sessionId (Primary Identifier)

- requesterId (Reference to Users)
- providerId (Reference to Users)
- skillId (Reference to Skills)
- scheduledTime
- status

Wallets

- walletId (Primary Identifier)
- userId (Reference to Users)
- balance

Transactions

- transactionId (Primary Identifier)
- walletId (Reference to Wallets)
- amount
- type
- sessionId (Optional reference to Sessions)

Messages

- messageId (Primary Identifier)
- senderId (Reference to Users)
- receiverId (Reference to Users)
- sessionId (Optional reference)
- content
- timestamp

Reviews

- reviewId (Primary Identifier)
- reviewerId (Reference to Users)
- reviewedUserId (Reference to Users)
- sessionId (Reference to Sessions)
- rating
- comment

Physical Schema (Firestore Implementation)

The system is implemented using Firebase Firestore, which stores data as collections of JSON-like documents.

Each collection contains documents identified by unique IDs, and relationships are handled using references (IDs) rather than joins.

Key characteristics of the physical schema include:

- Data is stored in **collections and documents**
- Each document contains key-value pairs
- Relationships are maintained using **document references (IDs)**
- Frequently accessed data (such as wallet balance) may be duplicated for faster reads
- Real-time updates are supported by Firestore synchronization

---

Normalization and Design Considerations

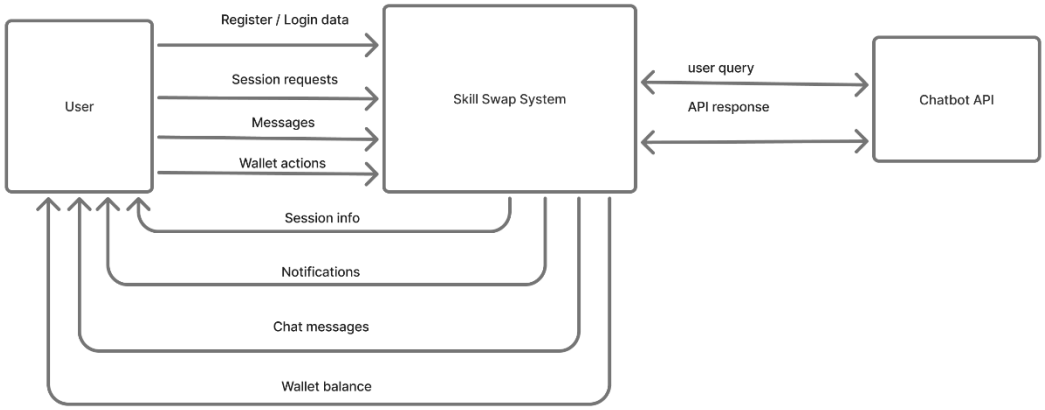
Although Firestore is a NoSQL database, basic normalization principles are applied to reduce redundancy and maintain data consistency.

- Core entities (Users, Sessions, Skills) are stored in separate collections
- Relationships are maintained through references instead of embedding large nested objects
- Data duplication is minimized, except in cases where performance optimization is required
- For example, wallet balance may be stored in both Users and Wallets collections to improve read performance
- Optional relationships (such as sessionId in Messages and Transactions) provide flexibility in the data model

This design balances normalization with performance, ensuring scalability and efficient real-time operations.

Context-Level Data Flow Diagram (Level 0) Description

Diagram



The context-level Data Flow Diagram (DFD) represents the Skill Swap System as a single high-level process that interacts with external entities.

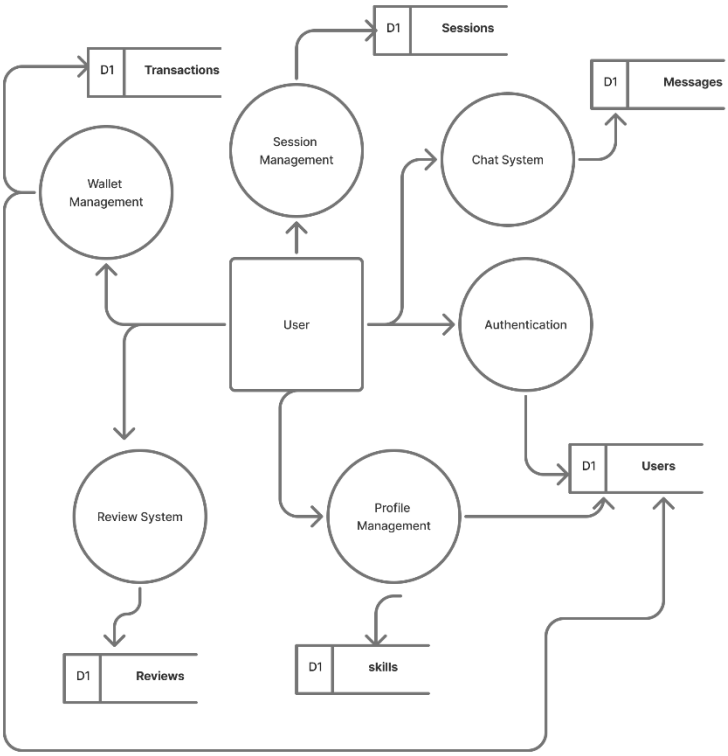
The primary external entity is the **User**, who interacts with the system by sending data such as registration and login information, session requests, messages, and wallet-related actions. In return, the system provides outputs including session information, notifications, chat messages, and wallet balance updates.

The system also interacts with an external **Chatbot API**, where user queries are sent to the API, and responses are returned to the system and then delivered to the user.

This diagram provides an overview of how data flows between the system and external components without showing internal processing details.

Detailed Data Flow Diagram (Level 1) Description

Diagram



The Level 1 Data Flow Diagram (DFD) expands the Skill Swap System into multiple internal processes, showing how data flows between system components and data stores.

The system is divided into several main processes:

- **Authentication Process:**
  Handles user registration and login. User data is validated and stored in the Users data store.
- **Profile Management Process:**
  Allows users to manage their personal information and skills. Data is stored and retrieved from the Users and Skills data stores.

- **Session Management Process:**
  Manages session requests, scheduling, and status updates. Session data is stored in the Sessions data store.
- **Wallet Management Process:**
  Handles wallet balance updates and point transactions. Data is stored in the Wallets and Transactions data stores.
- **Chat System Process:**
  Enables communication between users. Messages are stored and retrieved from the Messages data store.
- **Review System Process:**
  Allows users to submit feedback after sessions. Reviews are stored in the Reviews data store.

Each process interacts with its corresponding data store to store and retrieve information as needed. The user interacts with all processes by sending inputs and receiving outputs such as confirmations, updates, and notifications.

The system also integrates with the **Chatbot API**, where user queries are sent to the external service and responses are returned to the system and then delivered to the user.

Overall, the Level 1 DFD provides a detailed view of how data moves within the system, illustrating the interaction between processes, data stores, and external entities.

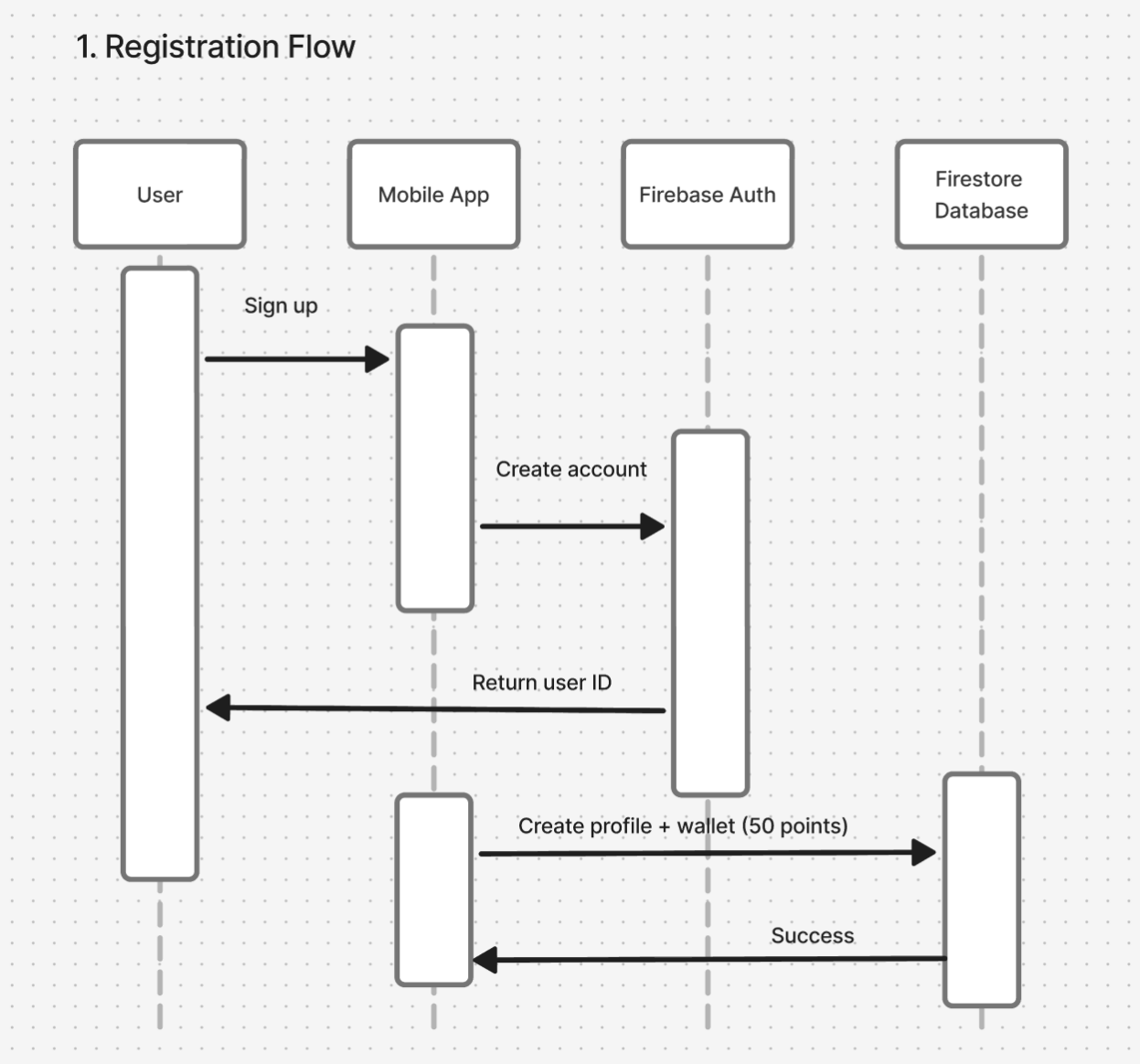
Sequence Diagrams Description

The system behavior is represented using multiple sequence diagrams that illustrate the interaction between users, the mobile application, and backend services. These diagrams describe the flow of operations for key system functionalities.

---

1. User Registration & Wallet Creation Flow

Diagram



This sequence describes how a new user registers in the system and receives an initialized wallet.

The process begins when the user submits registration details through the mobile application. The application sends the request to Firebase Authentication to create a new account. After successful account creation, a unique user ID is returned to the application.

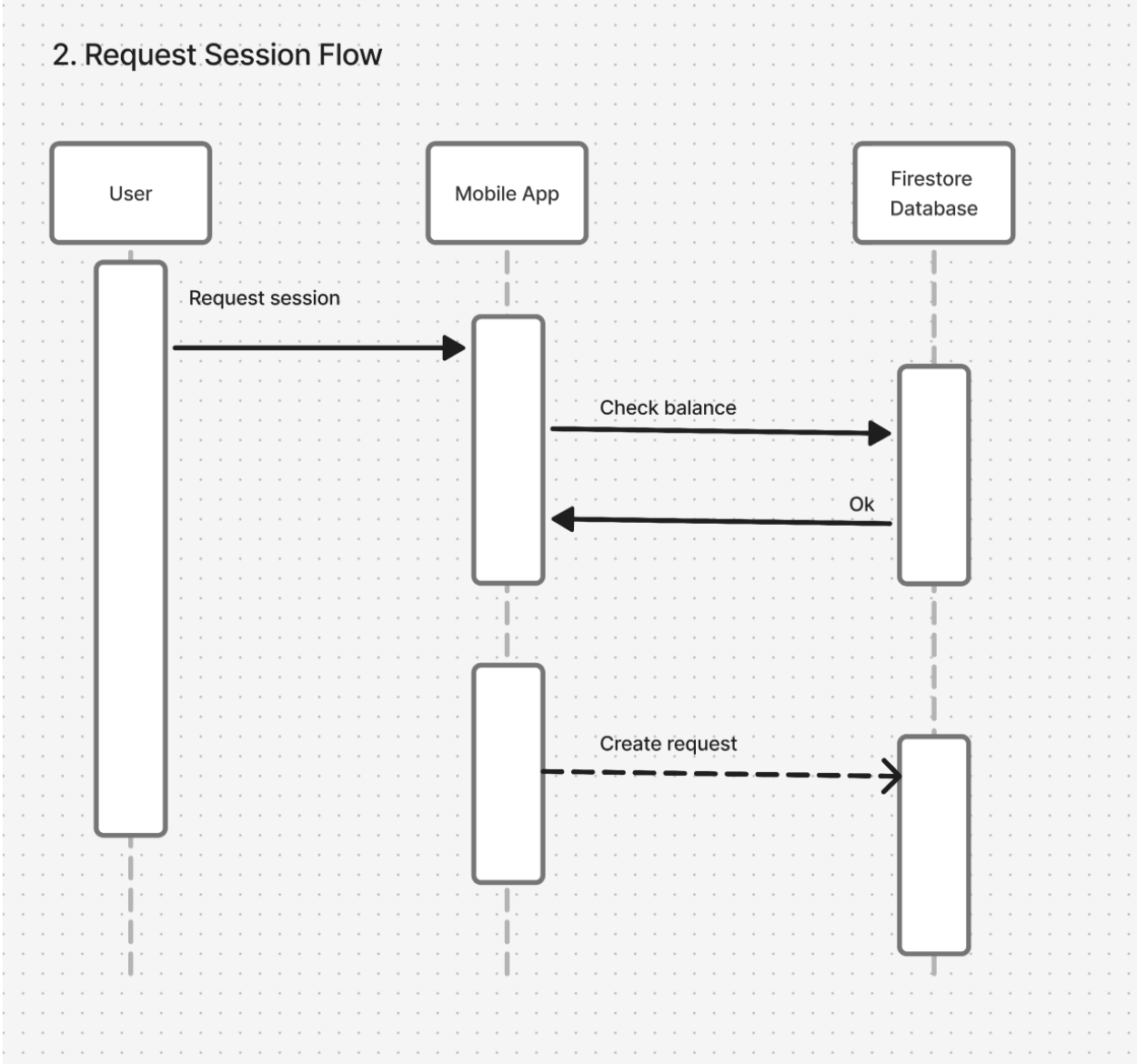
The application then creates a user profile in the database and initializes a Time Wallet with a default balance. Once the data is successfully stored, the user is redirected to the dashboard.

This flow ensures that every new user starts with a valid account and an initialized wallet ready for transactions.

---

2. Request Session Flow (Skill Exchange Request)

Diagram



This sequence illustrates how a user requests a learning session.

The learner selects a skill, a provider, and a preferred time through the application. The application checks the learner's wallet balance in the database to ensure sufficient points are available.

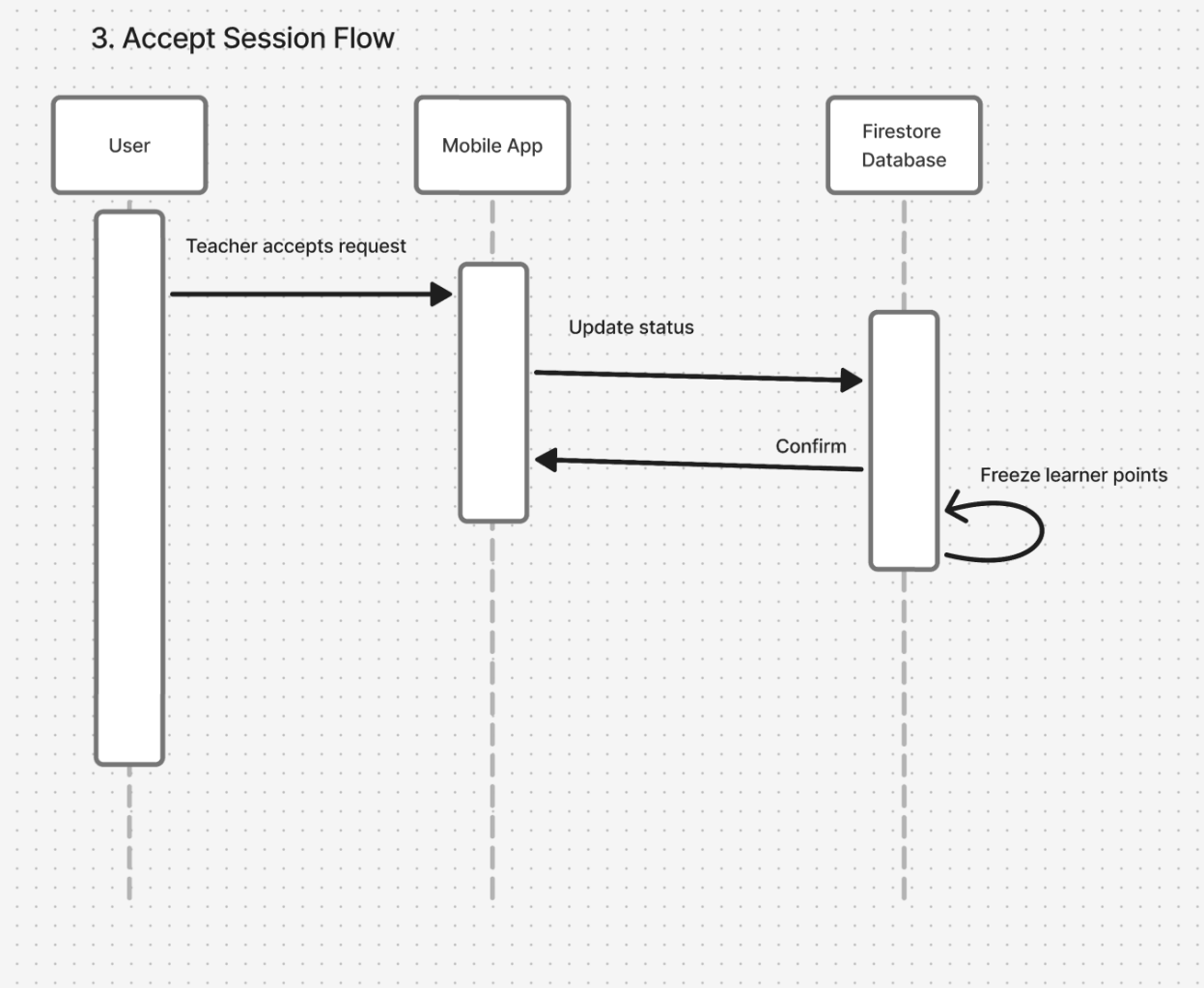
If the balance is insufficient, the process is terminated. Otherwise, a new session request is created with a pending status. The system then sends a notification to the provider informing them of the request.

This flow ensures that session requests are only created when the learner has enough points.

---

3. Accept Session & Wallet Lock Flow

Diagram



This sequence represents the provider's response to a session request.

When the provider accepts the request, the application updates the session status to accepted. At this stage, the system temporarily locks (freezes) the required points from the learner's wallet to prevent misuse.

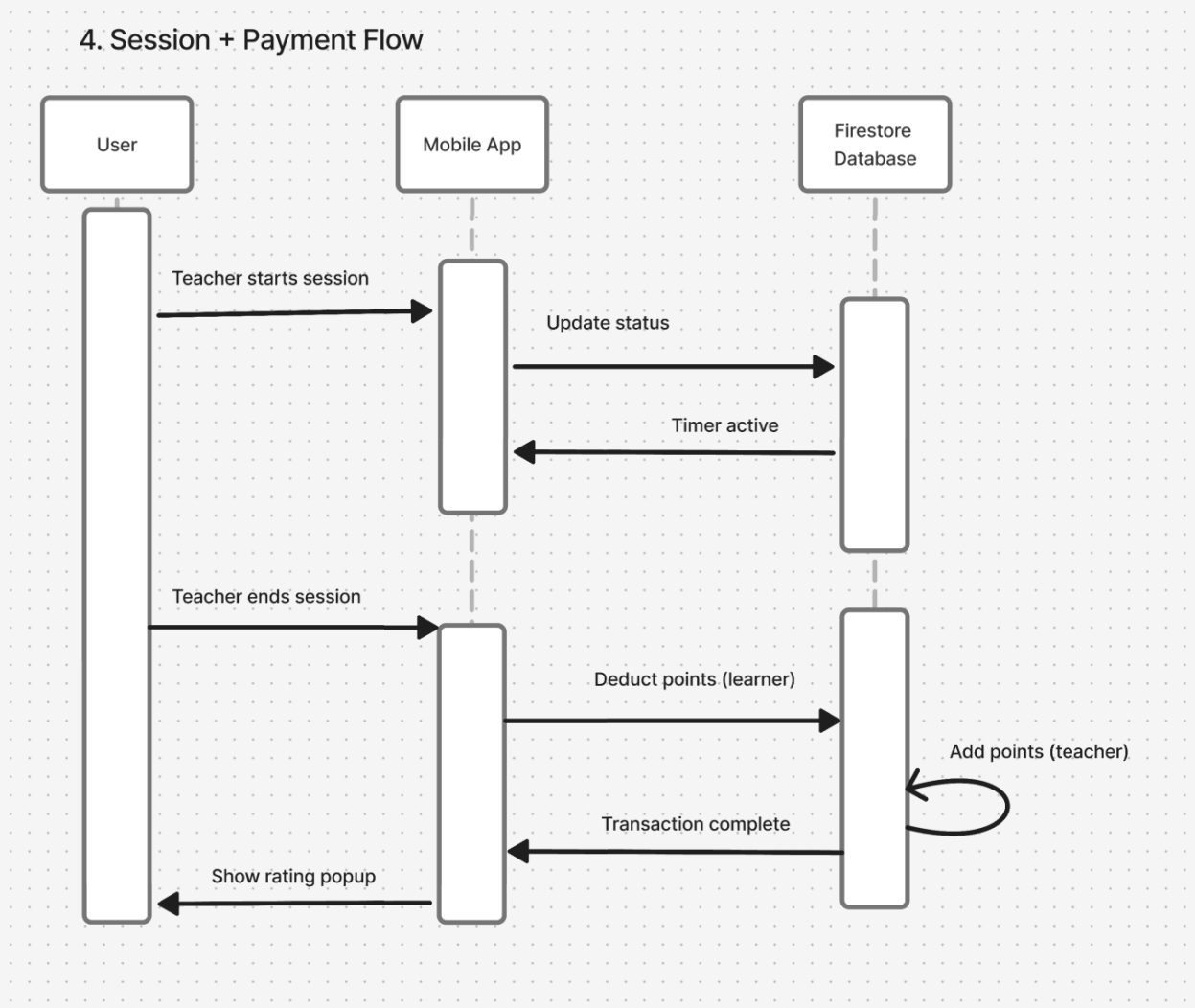
A notification is then sent to the learner confirming the acceptance, and both users are informed that the session has been successfully scheduled.

This mechanism ensures fairness and guarantees that payment is secured before the session begins.

---

5. Session Execution & Payment Transfer Flow

Diagram



This sequence describes how a session is conducted and how points are transferred.

The provider starts the session through the application, and the system updates the session status to ongoing. The application tracks the session duration.

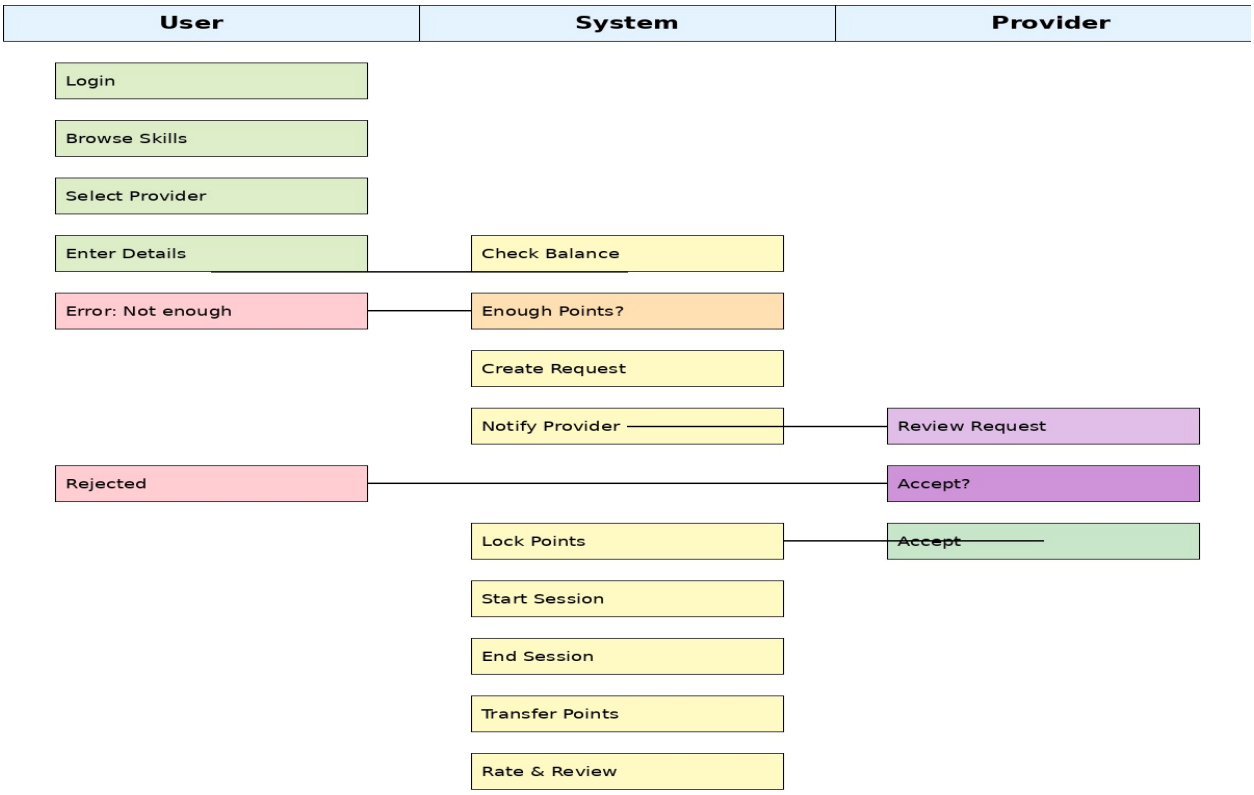
Once the session ends, the application triggers the payment process. The system deducts the points from the learner's wallet and transfers them to the provider's wallet. Both users' balances are updated accordingly.

After the transaction is completed, the system prompts both users to provide feedback.

This flow ensures accurate tracking of sessions and secure transfer of points.

Activity Diagram Description

Diagram



The Activity Diagrams illustrate the workflow of key system processes and user interactions within the Skill Swap platform. These diagrams represent the sequence of actions, decision points, and system responses.

1. Session Booking and Execution Flow

This activity diagram represents the complete lifecycle of a learning session.

The process starts when the user logs into the application and browses available skills. The user selects a teacher and preferred time, after which the system checks the user's wallet balance.

If the balance is insufficient, the process is terminated. Otherwise, a session request is created and sent to the teacher. The system then waits for the teacher's response.

If the request is rejected, the process ends. If accepted, the system temporarily locks the required points. The session is then conducted, and upon completion, the points are transferred from the learner to the teacher.

Finally, both users are prompted to rate the session, and the process ends.

---

2. User Registration and Wallet Initialization

This diagram describes the process of creating a new account.

The workflow begins when the user opens the application and submits registration details. The system creates an account using the authentication service, followed by creating a user profile in the database.

A wallet is then initialized with a default number of points. After successful setup, the user is redirected to the dashboard.

---

3. Points Purchase Flow

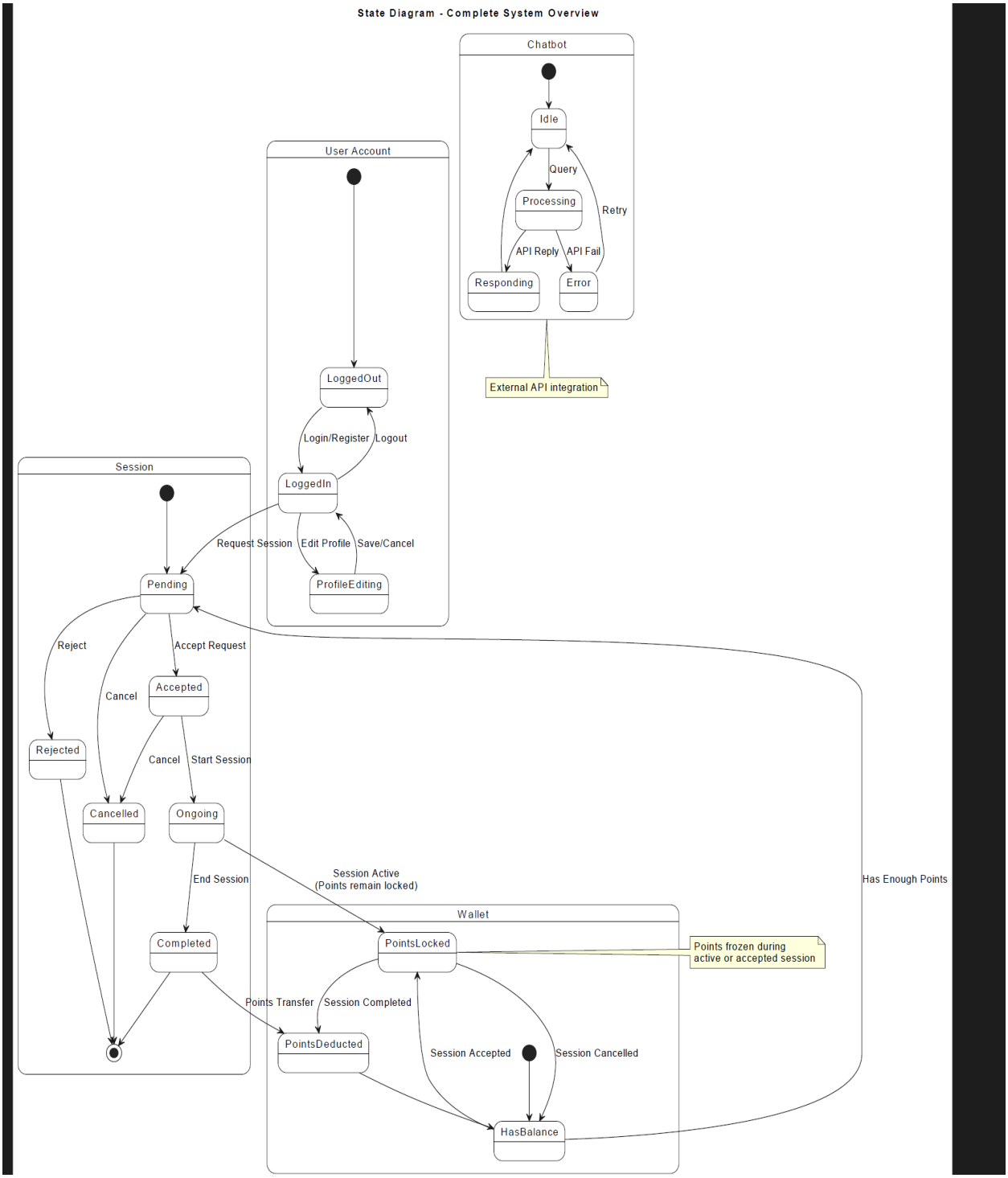
This diagram represents how users can add points to their wallet.

The process starts when the user accesses the wallet section and selects the option to purchase points. The user enters the desired amount, and the system processes the payment (or simulated transaction).

Once the payment is successful, the wallet balance is updated, and a confirmation message is displayed.

State Diagram Description

Diagram



This state diagram represents the dynamic behavior of the Skill Swap system, illustrating how different components (User Account, Session, Wallet, and Chatbot) transition between states based on user actions and system events.

---

1. User Account States

The **User Account** component models the authentication and profile lifecycle of a user.

- The system starts in the **LoggedOut** state.
- The user can transition to **LoggedIn** through the *Login/Register* action.
- From **LoggedIn**, the user can:
    - Edit their profile → moves to **ProfileEditing**.
    - Save or cancel changes → returns to **LoggedIn**.
    - Logout → returns to **LoggedOut**.
- While logged in, the user can also initiate session requests.

---

2. Session States

The **Session** component represents the lifecycle of a skill exchange session.

- A session begins in the **Pending** state after a request is created.
- From **Pending**, the session can:
    - Be **Accepted** by the teacher.
    - Be **Rejected**.
    - Be **Cancelled** by the requester.
- If accepted:
    - The session transitions to **Accepted**, then to **Ongoing** when it starts.
    - After completion, it moves to **Completed**.
- Any rejected or cancelled sessions terminate the flow.

This flow ensures controlled interaction between users and proper handling of session states.

---

3. Wallet (Points System) States

The **Wallet** manages user points during session interactions.

- Initially, the wallet is in **HasBalance** state.
- When a session is accepted:
    - Points are moved to **PointsLocked** (frozen state).
    - This ensures the learner cannot spend these points elsewhere.

- During the session:
  - Points remain locked (**Session Active**).
- After session completion:
  - Points are transferred → **PointsDeducted**.
  - Then the wallet returns to **HasBalance**.
- If the session is cancelled:
  - Locked points are released back to **HasBalance**.

This mechanism guarantees fairness and prevents misuse of the system.

---

4. Chatbot States (External API Integration)

The **Chatbot** operates as an external integrated service.

- Starts in **Idle** state.
- When the user sends a query:
  - Moves to **Processing**.
- Based on API response:
  - If successful → transitions to **Responding**.
  - If failure occurs → transitions to **Error**.
- From **Error**, the system can retry and return to **Processing**.

This component demonstrates integration with an external API (e.g., Gemini), ensuring intelligent assistance within the app.

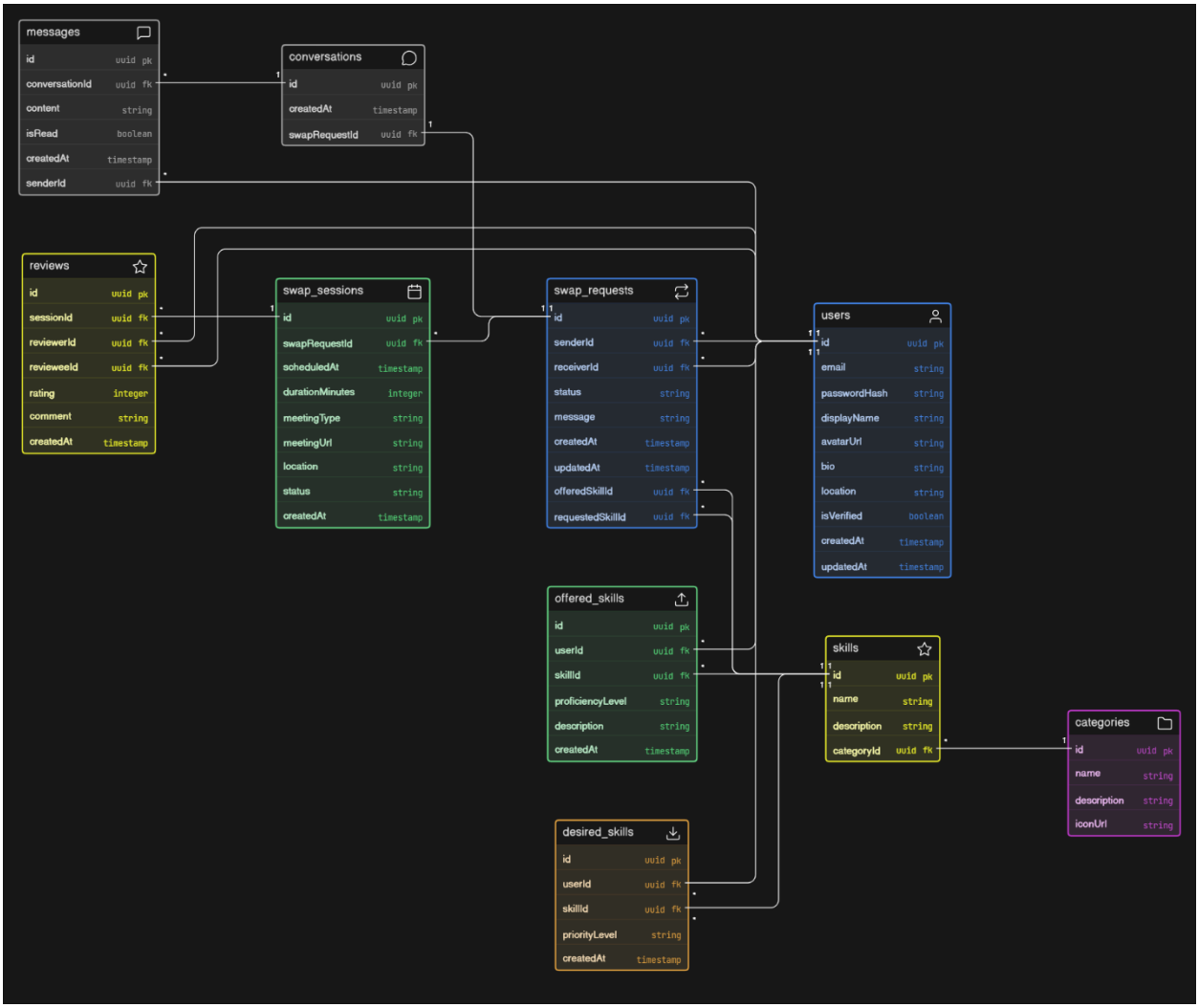
---

5. System Interaction Overview

- The user must be **LoggedIn** and have sufficient balance (**Has Enough Points**) to request a session.
- Session acceptance triggers wallet locking.
- Session completion triggers point transfer.
- The chatbot operates independently but supports user interaction at any time.

Class Diagram Description

Diagram



The class diagram represents the structural design of the Skill Swap system, illustrating the main entities, their attributes, and the relationships between them. The system is centered around users who interact with each other by offering skills, requesting sessions, and exchanging knowledge.

The **User** entity is the core of the system, storing essential information such as email, password, display name, profile details, and account status. Each user can offer skills and also express interest in learning new ones.

Skills are organized using two main entities: **Skills** and **Categories**. Each skill belongs to a category, allowing the system to group related skills for easier browsing and filtering. Users can associate themselves with skills in two ways: through the **Offered Skills** entity, which represents

skills a user can teach, and the **Desired Skills** entity, which represents skills a user wants to learn.

The interaction between users is managed through the **Swap Requests** entity. A swap request is created when one user (sender) requests a session with another user (receiver), specifying the offered and requested skills. Each request includes a status to track its progress, such as pending, accepted, or rejected.

Once a request is accepted, it leads to the creation of a **Swap Session**, which contains details about the scheduled time, duration, meeting type (online or offline), and session status. This entity represents the actual execution of the skill exchange.

Communication between users is handled through the **Conversations** and **Messages** entities. Each conversation is linked to a specific swap request, and multiple messages can be exchanged within a conversation, allowing users to coordinate session details.

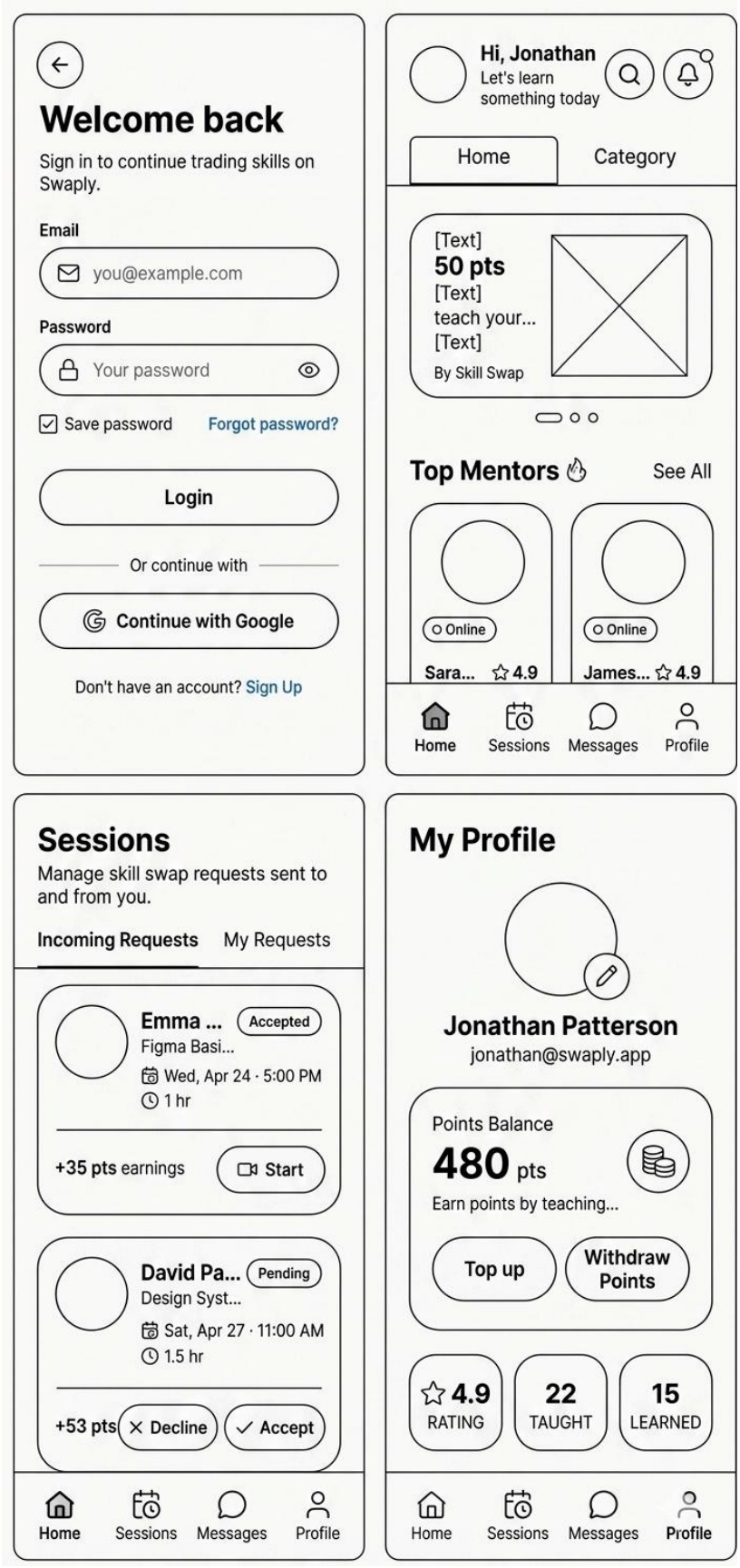
After completing a session, users can provide feedback through the **Reviews** entity. Reviews include a rating and a comment, and are linked to both the session and the users involved, helping build trust and reputation within the platform.

Overall, the diagram demonstrates how the system integrates user profiles, skill management, session scheduling, communication, and feedback into a cohesive structure that supports peer-to-peer skill exchange.

---

4. UI/UX Design & Prototyping

Wireframes & Mockups


Welcome back
Sign in to continue trading skills on Swaply.
Email
you@example.com
Password
Your password
Save password
Forgot password?
Login
Or continue with
Continue with Google
Don't have an account? Sign Up
Hi, Jonathan
Let's learn something today
Home
Category
[Text]
50 pts
[Text]
teach your...
[Text]
By Skill Swap
Top Mentors
See All
Online
Sara... 4.9
Online
James... 4.9
Home
Sessions
Messages
Profile
Sessions
Manage skill swap requests sent to and from you.
Incoming Requests
My Requests
Emma ...
Accepted
Figma Basi...
Wed, Apr 24 · 5:00 PM
1 hr
+35 pts earnings
Start
David Pa...
Pending
Design Syst...
Sat, Apr 27 · 11:00 AM
1.5 hr
+53 pts
Decline
Accept
Home
Sessions
Messages
Profile
My Profile
Jonathan Patterson
jonathan@swaply.app
Points Balance
480 pts
Earn points by teaching...
Top up
Withdraw Points
4.9
RATING
22
TAUGHT
15
LEARNED
Home
Sessions
Messages
Profile

UI/UX Guidelines

1. Design Principles

The application follows a **minimal, modern, and user-centered design approach** to ensure ease of use and visual appeal. The interface emphasizes simplicity by reducing clutter and focusing on key actions such as account creation and login. Clear visual hierarchy is maintained through the use of spacing, color contrast, and typography, guiding users naturally through the interface.

Consistency is applied across all screens in terms of layout, button styles, and color usage, ensuring a smooth and predictable user experience. The design also prioritizes clarity, making primary actions (e.g., “Create Account”) more prominent than secondary actions.

---

2. Color Scheme

The application uses a **gradient-based purple theme** to create a modern and engaging look while maintaining readability.

- **Primary Color:** Purple gradient (used for backgrounds and primary buttons)
- **Secondary Color:** White (#FFFFFF) for cards and surfaces
- **Accent Color:** Light purple / soft glow for highlights
- **Text Colors:**
    - White for text on dark backgrounds
    - Dark gray/black for text on light surfaces

The color contrast is carefully balanced to ensure that important elements, such as call-to-action buttons, stand out clearly.

---

3. Typography

The typography is clean and readable, supporting both usability and visual hierarchy.

- **Font Style:** Sans-serif (modern and easy to read)
- **Headings:** Bold and larger size to emphasize key messages
- **Body Text:** Medium weight for readability
- **Buttons:** Bold text to highlight interactivity

Proper spacing between lines and elements improves readability and prevents visual clutter.

---

4. Layout and Components

The layout follows a **centered and balanced structure**, especially for onboarding screens:

- Key elements (logo, tagline, buttons) are centrally aligned
- Rounded cards and buttons create a friendly and modern feel
- Primary button is large and highly visible
- Secondary actions are less visually dominant

Spacing and padding are used effectively to separate sections and improve user focus.

---

5. Accessibility Considerations

The design incorporates accessibility best practices to ensure usability for a wide range of users:

- **Color Contrast:** Sufficient contrast between text and background for readability
- **Button Size:** Large clickable areas for easy interaction
- **Readable Fonts:** Clear and legible typography for all screen sizes
- **Simple Navigation:** Minimal steps required to complete actions
- **Visual Hierarchy:** Helps users quickly understand where to focus

---

6. User Experience (UX) Focus

The UI is designed to support a smooth onboarding experience:

- Users immediately understand the app’s purpose through the tagline
- The main action (“Create Account”) is clearly highlighted
- Alternative action (“I already have an account”) is easily accessible
- Clean design reduces cognitive load and improves engagement

---

5. System Deployment & Integration

Technology Stack

The system is built using a modern and scalable technology stack that ensures high performance, real-time interaction, and cross-platform compatibility.

1. Frontend

The frontend of the application is developed using **Flutter**, a cross-platform UI toolkit. Flutter enables the creation of a single codebase that runs efficiently on both Android and iOS devices. It provides a rich set of customizable widgets, allowing the application to deliver a consistent and responsive user interface.

2. Backend

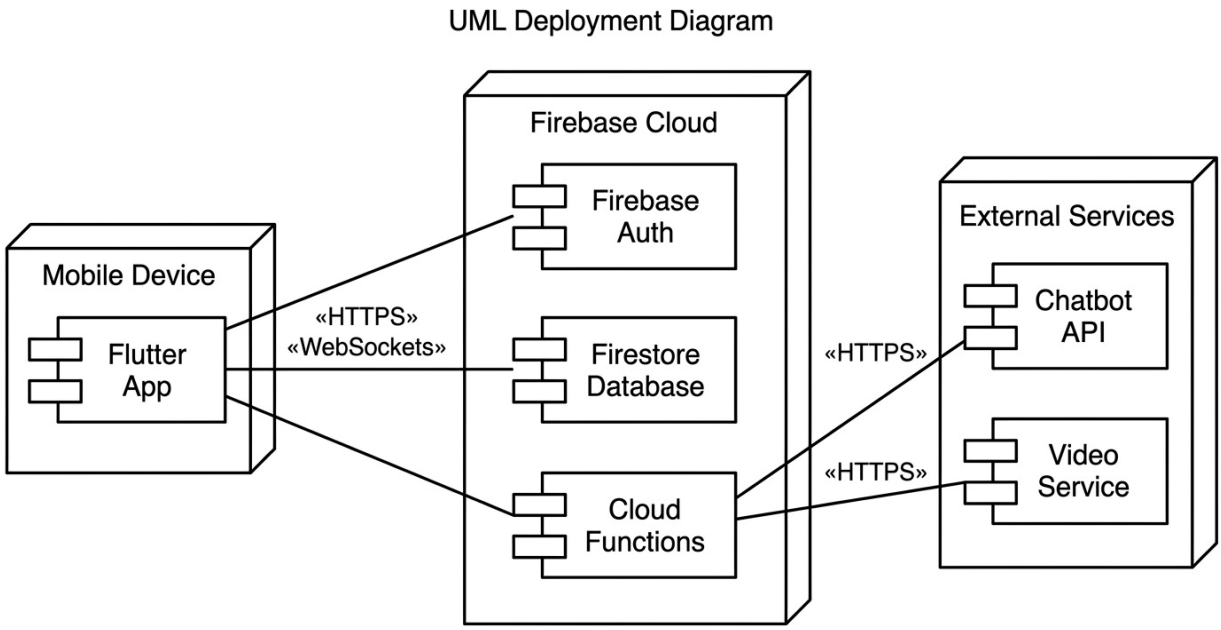
The backend is powered by **Firebase**, a cloud-based platform that provides ready-to-use backend services. Firebase handles essential functionalities such as authentication, cloud storage, and real-time communication. This reduces development complexity and ensures scalability and reliability.

3. Database

The application uses **Cloud Firestore**, a NoSQL, real-time database provided by Firebase. Firestore stores data in collections and documents, enabling flexible and efficient data management. It supports real-time synchronization, allowing users to see updates instantly, which is essential for features like messaging, session updates, and notifications.

Deployment Diagram Description

Diagram



The deployment diagram illustrates the physical architecture of the Skill Swap system and shows how software components are distributed across different nodes, including the mobile device, cloud services, and external systems.

The system is primarily divided into three main layers: the mobile client, the cloud backend, and external service integrations.

The **mobile device** hosts the Flutter application, which acts as the user interface. It is responsible for handling user interactions, displaying data, and sending requests to the backend services. All communication between the mobile application and the cloud backend is performed securely using HTTPS and, where needed, WebSockets for real-time updates.

The **Firebase Cloud** serves as the core backend infrastructure. It contains several key components:

- **Firebase Authentication**, which manages user registration, login, and secure identity verification.
- **Cloud Firestore Database**, which stores all application data such as user profiles, sessions, messages, and wallet transactions.
- **Cloud Functions**, which handle backend logic, including session management, point calculations, and triggering automated processes such as notifications.

These components work together to ensure scalability, real-time data synchronization, and secure data handling.

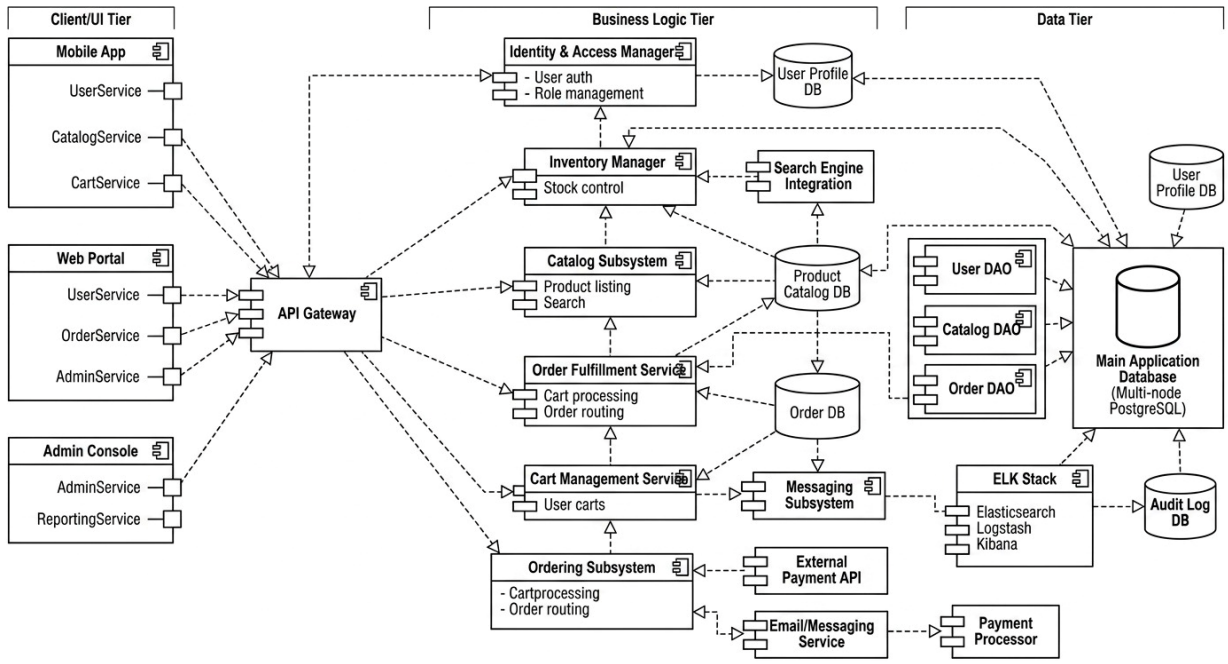
In addition to the core backend, the system integrates with **external services** through secure HTTPS communication. These include:

- A **Chatbot API**, which provides automated assistance and user support داخل التطبيق.
- A **Video Service**, which enables real-time communication between users during learning sessions.

Overall, the diagram demonstrates a client–server architecture where the mobile application interacts with cloud-based backend services and external APIs. This design ensures flexibility, scalability, and efficient communication between all system components.

Component Diagram Description

Diagram



The component diagram illustrates the high-level structure of the system by showing its main components, their responsibilities, and the interactions between them across different

architectural layers. The system follows a layered architecture consisting of the Client/UI Tier, Business Logic Tier, and Data Tier.

In the **Client/UI Tier**, the system provides multiple interfaces, including the mobile application, web portal, and admin console. Each interface contains specialized services such as user management, catalog browsing, cart handling, and administrative operations. These client-side components communicate with the backend through a centralized API Gateway, which acts as the main entry point for all requests.

The **API Gateway** is responsible for routing incoming requests to the appropriate backend services. It simplifies communication between the frontend and backend while ensuring scalability and maintainability.

The **Business Logic Tier** contains the core functional components of the system. These include identity and access management for authentication and role handling, inventory management for stock control, and the catalog subsystem for managing product listings and search functionality. The order fulfillment and cart management services handle the processing of user carts and order execution, while the ordering subsystem manages overall order routing and coordination.

Additional components within this layer include the messaging subsystem for internal communication between services and the integration with external systems such as payment APIs and email/messaging services. These integrations enable features like online payments and user notifications.

The **Data Tier** is responsible for persistent data storage. It includes multiple specialized databases such as user profile, product catalog, and order databases, in addition to a centralized main application database. Data access is managed through dedicated Data Access Objects (DAOs), which abstract database operations and ensure a clean separation between business logic and data handling.

The system also includes supporting components such as logging and monitoring tools (ELK Stack) and an audit log database to track system activities and ensure reliability and traceability.

Overall, the diagram demonstrates a modular and scalable system design where components are clearly separated by responsibility, enabling efficient communication, maintainability, and extensibility.

---

6. Additional Deliverables

API Documentation

The system integrates external APIs to extend its functionality, particularly for the chatbot feature. The chatbot is powered by the **Google Gemini API**, which enables intelligent conversational responses داخل التطبيق.

Chatbot API Endpoint

Endpoint Name: Send Message to Chatbot
Method: POST
URL: `/api/chatbot/message`

Request Body

```
{
  "userId": "string",
  "message": "string"
}
```

Response

```
{
  "reply": "string",
  "timestamp": "datetime"
}
```

Description

- The user sends a message from the mobile application.
- The backend (Firebase Cloud Function) forwards the request to the Gemini API.
- The API processes the message and returns a response.
- The response is sent back to the user interface.

Authentication

- API keys are securely stored in backend environment variables.
- Direct access from the client is restricted to ensure security.

Testing & Validation

To ensure system reliability and performance, multiple levels of testing are applied:

1. Unit Testing

- Each module is tested independently.
- Examples:
  - Authentication functions
  - Wallet calculations
  - Session creation logic
- Tools: Flutter test framework & Firebase emulator

2. Integration Testing

- Tests interactions between components.
- Examples:
  - Chatbot API integration with backend
  - Session booking with wallet deduction
  - Firebase Authentication with Firestore data

3. User Acceptance Testing (UAT)

- Real users test the system to validate usability.
- Scenarios include:
  - Creating an account
  - Booking a session
  - Using the chatbot
  - Completing a session and rating

Validation Criteria

- System responds within acceptable time (< 2 seconds for most operations)
- No critical errors during main user flows
- Accurate data handling (points, sessions, messages)

Deployment Strategy

The system is deployed using a cloud-based architecture to ensure scalability and availability.

Hosting Environment

- Backend services are hosted on **Firebase Cloud**
- Database is managed using **Cloud Firestore**
- Mobile application is deployed on Android devices (and optionally iOS)

Deployment Pipeline

- Code is developed locally using Flutter
- Backend functions are deployed using Firebase CLI
- Continuous updates are applied through version control (e.g., Git)

Steps:

1. Build Flutter application
2. Deploy Firebase Cloud Functions
3. Configure Firestore database
4. Connect app to backend services
5. Release application

Scaling Considerations

- Firebase automatically scales based on user demand
- Firestore supports real-time updates and high concurrency
- Cloud Functions handle increasing request loads dynamically

Security Considerations

- Secure API keys (Gemini API) using environment variables
- Firebase Authentication for user identity
- Firestore security rules to protect data access